

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

Preparing Global Cybersecurity Leaders Through Excellence in Hands-On Training

LAB REPORT

Memory Forensics with Volatility:

Investigation of a Data Exfiltration and Persistence Incident on a Windows 7 Host

Field	Detail
Lab Title	Memory Forensics with Volatility
Student Name	Fuseini Imoru Kantuogaa
Student ID	C11/26/DFIT/17291
Course Name	Case Study 3 — Illegal File Transferring: Memory, USB and User-Activity Forensics
Instructor Name	Mr. Amin Iddrisu
Date of Submission	17/09/2026
Version	Version 1.0

1. Executive Summary

This lab examined a 1 GB physical memory image acquired from a Windows 7 host, using both the current Volatility 3 framework and, for plugins not yet ported, the legacy Volatility 2 framework. The objective was to establish what the host was, whether it showed signs of compromise, and, if so, to reconstruct what an intruder or insider had done on it.

The image was identified as a 32-bit Windows 7 Service Pack 1 virtual machine named IE8WIN7, running under Oracle VirtualBox on a host with an Intel Core i7-7700 processor. Registry analysis revealed a second, non-standard local account, `sshd_server`, and an installed OpenSSH server running as a Windows service. Command-history recovery from the console subsystem showed the primary user, `IEUser`, running `ipconfig`, navigating to a Downloads folder, and copying a file named `secret_file.docx` to drive F:, a USB mass-storage device that registry and browser artefacts confirm was attached to the system at the time. Independent string searches and a recovered Chrome browsing history both show repeated navigation to `http://192.168.56.5/secret_file.docx`, indicating the same file was also transferred across the network to a second host on the internal 192.168.56.0/24 range.

A local password hash was recovered with `hashdump` and cracked offline with John the Ripper, confirming the account password. The combination of an unauthorised local account, an installed remote-access service, and two independent, corroborating channels of file movement supports a finding that this host was used to stage and exfiltrate a specific document, most likely `secret_file.docx`, both to removable media and across the local network.

2. Lab Objectives

The objectives of this lab were as follows:

- To acquire a physical memory image from an external repository and confirm it loads correctly under both Volatility 3 and Volatility 2.
- To identify the operating system, build, host name and virtualisation platform of the imaged machine from kernel and registry artefacts.
- To enumerate local user accounts and registry hives, and to identify any account or service that does not belong to a standard installation.

- To recover command-line history and console activity to reconstruct what the interactive user actually typed.
- To identify removable and network storage activity connected to a specific file of interest, and to corroborate that activity across independent evidence sources.
- To recover and, where feasible, crack local password hashes as a further line of attribution.
- To document the investigative process, including the limitations of the tooling used, to a standard that would support a coherent incident report.

3. Tools and Resources Used

All analysis was performed on a Kali Linux host. Both generations of the Volatility framework were used, because a number of legacy plugins relied on in this lab have not yet been ported to Volatility 3.

Tool / Resource	Purpose and relevance
Kali Linux	Analysis platform hosting all tools used in this lab.
Volatility 3 (vol.py, framework 2.28.2)	Current memory-forensics framework. Used for windows.info, the initial and most reliable identification of the imaged operating system.
Volatility 2 (vol.py, framework 2.6.1)	Legacy framework, built from source, used for plugins not yet available in Volatility 3: imageinfo, printkey, hivelist, netscan, pslist, cmdscan, consoles, hashdump, iehistory and shellbags.
wget	Retrieval of the memdumpWin7.mem evidence file from a Dropbox share.
strings / grep / awk	Text extraction and filtering of raw memory content and command output, used to locate IP

Tool / Resource	Purpose and relevance
	addresses, file paths and URLs not exposed by a specific plugin.
John the Ripper	Offline recovery of a plaintext password from an NTLM hash recovered by hashdump.
chrome_ragamuffin (third-party Volatility 2 plugin)	Community plugin used to recover Chrome browser artefacts, including DOM and navigation history, directly from process memory.
git	Retrieval of the Volatility 2 source tree and the chrome_ragamuffin plugin from GitHub.
memdumpWin7.mem	Primary evidence file: a 1,073,676,288-byte (1024 MB) physical memory capture of a Windows 7 SP1 x86 virtual machine.

4. Methodology

The work proceeded in five phases: environment preparation, evidence acquisition, host identification, account and persistence analysis, and activity reconstruction. Because several plugins needed for this lab depend on packages not available for the pre-built Volatility 2 binary, the legacy framework was built from source, and its plugin import failures were noted rather than treated as blocking, since the affected plugins (mac/registry decryption features requiring PyCrypto and distorm3) were not required for this investigation.

4.1 Phase One — Environment Preparation

1. Reviewed the Volatility 3 command-line interface with `vol.py -h` to confirm available global options (Figures 1 and 2).

2. Cloned the Volatility 2 source tree from GitHub and ran `python2 vol.py --info`. Several plugins failed to import due to missing `Crypto.Hash` and `distorm3` modules; this was recorded and the remaining, unaffected plugin set was confirmed available (Figures 3 to 5).

4.2 Phase Two — Evidence Acquisition

3. Retrieved the evidence file, `memdumpWin7.mem`, from a Dropbox share using `wget`, following the redirect chain to the final content host (Figures 6 and 7).
4. Confirmed the completed download at 1,073,676,288 bytes with `ls -l` (Figure 8).

4.3 Phase Three — Host Identification

5. Ran `windows.info` under Volatility 3, which automatically retrieved the matching kernel PDB symbols and reported the kernel base, DTB, and a 32-bit PAE Windows 7 SP1 (build 7601) system with a recorded `SystemTime` of 2019-01-06 15:09:06 UTC (Figures 9 to 11).
6. Cross-checked this identification with the legacy `imageinfo` plugin, which independently suggested the `Win7SP1x86` profile family from the KDBG structure (Figure 12).
7. Used `printkey` to inspect the `SOFTWARE` hive at `\Users\IEUser\ntuser.dat` and confirm the primary account, `IEUser`, and the host name `IE8WIN7` (Figures 13 and 14).
8. Queried the `HARDWARE` hive to identify the underlying platform: an Intel Core i7-7700 CPU exposed by Oracle VM VirtualBox 6.0.0 (Figures 20 to 22).
9. Confirmed the NetBIOS computer name `IE8WIN7` directly from the `SYSTEM` hive (Figure 27).

4.4 Phase Four — Account and Persistence Analysis

10. Enumerated the `ProfileList` key in the `SOFTWARE` hive and identified two custom local accounts beyond the built-in defaults: `SID ...-1000` (`IEUser`) and `SID ...-1002`, whose `ProfileImagePath` resolves to `C:\Users\sshd_server` (Figures 15 to 17).
11. Confirmed `AutoAdminLogon` was enabled for `IEUser` in the `Winlogon` key, consistent with an unattended test image (Figure 18).

12. Ran hivelist to confirm that separate ntuser.dat and UsrClass.dat hives exist on disk for both IEUser and sshd_server, showing the second account had actually logged on and built a profile, not merely been created (Figure 19).
13. Ran netscan to enumerate listening TCP sockets, noting standard Windows services alongside a socket bound to the host's own address, 192.168.56.8, on port 139 (Figure 28).
14. Ran hashdump against the SAM hive, recovering NTLM hashes for five accounts: Administrator, Guest, IEUser, and two further non-standard accounts, sshd and sshd_server (Figure 56).
15. Located a local password wordlist, seeded it with a candidate password, and ran John the Ripper against the recovered IEUser/Administrator hash, recovering the plaintext value Passw0rd! in under one second (Figures 57 to 61).

4.5 Phase Five — Activity Reconstruction

16. Ran cmdscan and consoles to recover command history attached to two conhost.exe processes. One was attached to sshd.exe (PID 2016), hosted by C:\Program Files\OpenSSH\bin\cygrunsrv.exe; the other was attached to an interactive cmd.exe session (Figures 33 to 36).
17. Recovered the full recorded command sequence from the cmd.exe session — ipconfig, cd Downloads, copy secret_file.docx F: — and replayed the associated screen buffer to confirm the shell's own "1 file(s) copied" response (Figures 36 to 38).
18. Cross-referenced the host's own IP address from the ipconfig output, 192.168.56.8, against the netscan results from phase four (Figure 37).
19. Searched the SYSTEM hive for USB storage evidence, finding a single USBSTOR device, Ven_General Prod_UDisk Rev_5.00, registered at 15:02:54 on the day of the incident, and confirmed its DeviceInstance and hardware ID strings (Figures 39 to 46).
20. Confirmed that drive letter F: was actually mounted within the IEUser session by locating the corresponding MountedDevices and MountPoints2 entries (Figures 47, 48 and 54).
21. Ran a raw string search of the entire memory image for the internal address 192.168.56.5, recovering browser artefacts referencing /secret_file.docx and the URL http://192.168.56.5/secret_file.docx (Figures 31 and 32).

22. Ran `iehistory` to check Internet Explorer's own cache, which returned only routine browsing artefacts and no hits for the year 2019, indicating the observed web activity did not go through Internet Explorer (Figures 49 and 50).
23. Generated a shellbags timeline filtered to the day of the incident, showing folder-view activity clustered between 14:32 and 15:07 UTC, consistent with the other timestamps recovered (Figures 51 and 52).
24. Built and loaded the third-party plugin `chrome_ragamuffin` to recover artefacts directly from Chrome process memory, and used a scripted string extraction to tabulate every Chrome URL and title referencing `secret_file.docx`, confirming repeated navigation to the external host both by direct URL and via a Google search (Figures 62 to 65).

5. Screenshots and Evidence

The figures below follow the order in which the work was performed: environment setup and acquisition (Figures 1–8), host identification (Figures 9–27), account and persistence analysis (Figures 28 and 39–61, interleaved with the registry evidence they build on), and activity reconstruction (Figures 31–38 and 62–65).

5.1 Environment Preparation and Acquisition (Figures 1–8)

```
(kali@kali)-[~/volatility3]
$ ./vol.py -h
Volatility 3 Framework 2.28.2
usage: vol.py [-h] [-c CONFIG] [--parallelism [{processes,threads,off}]] [-e EXTEND] [-p PLUGIN_DIRS]
             [-s SYMBOL_DIRS] [-v] [-l LOG] [-o OUTPUT_DIR] [-q] [-f FILE] [--write-config]
             [--save-config SAVE_CONFIG] [--clear-cache] [--cache-path CACHE_PATH] [--offline | -u URL]
             [--filters FILTERS] [--hide-columns [HIDE_COLUMNS ...]] [-r RENDERER]
             [--single-location SINGLE_LOCATION] [--stackers [STACKERS ...]]
             [--single-swap-locations [SINGLE_SWAP_LOCATIONS ...]]
             PLUGIN ...

An open-source memory forensics framework

options:
  -h, --help                Show this help message and exit, for specific plugin options use 'vol.py <pluginname>
                           --help'
  -c, --config CONFIG        Load the configuration from a json file
  --parallelism [{processes,threads,off}]
                           Enables parallelism (defaults to off if no argument given)
  -e, --extend EXTEND        Extend the configuration with a new (or changed) setting
  -p, --plugin-dirs PLUGIN_DIRS
                           Semi-colon separated list of paths to find plugins
  -s, --symbol-dirs SYMBOL_DIRS
                           Semi-colon separated list of paths to find symbols
  -v, --verbosity            Increase output verbosity
```

Figure 1: Volatility 3 (vol.py) top-level help output, showing the framework version (2.28.2) and the general command-line options available.

```
-l, --log LOG                Log output to a file as well as the console
-o, --output-dir OUTPUT_DIR  Directory in which to output any generated files
-q, --quiet                 Remove progress feedback
-f, --file FILE              Shorthand for --single-location=file:// if single-location is not defined
--write-config               Write configuration JSON file out to config.json
--save-config SAVE_CONFIG    Save configuration JSON file to a file
--clear-cache                Clears out all short-term cached items
--cache-path CACHE_PATH      Change the default path (/home/kali/.cache/volatility3) used to store the cache
--offline                    Do not search online for additional JSON files
-u, --remote-isf-url URL     Search online for ISF json files
--filters FILTERS            List of filters to apply to the output (in the form of [+~]columnname,pattern[!])
--hide-columns [HIDE_COLUMNS ...]
                           Case-insensitive space separated list of prefixes to determine which columns to hide
                           in the output if provided
-r, --renderer RENDERER      Determines how to render the output (quick, none, csv, pretty, json, jsonl, mermaid)
--single-location SINGLE_LOCATION
                           Specifies a base location on which to stack
--stackers [STACKERS ...]    List of stackers
--single-swap-locations [SINGLE_SWAP_LOCATIONS ...]
```

Figure 2: Continuation of the Volatility 3 help output, documenting output, logging, filtering and rendering options including `--renderer` and `--single-location`.

```
(kali㉿kali)-[~/volatility3]
$ git clone https://github.com/volatilityfoundation/volatility.git
ls volatility
cd volatility
Cloning into 'volatility' ...
remote: Enumerating objects: 27414, done.
remote: Counting objects: 100% (2/2), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 27414 (delta 0), reused 0 (delta 0), pack-reused 27412 (from 2)
Receiving objects: 100% (27414/27414), 21.11 MiB | 2.17 MiB/s, done.
Resolving deltas: 100% (19758/19758), done.
AUTHORS.txt  CREDITS.txt  Makefile      pyinstaller      resources  volatility
CHANGELOG.txt LEGAL.txt    MANIFEST.in  pyinstaller.spec setup.py   vol.py
contrib      LICENSE.txt  PKG-INFO     README.txt      tools

(kali㉿kali)-[~/volatility3/volatility]
$ python2 vol.py --info
Volatility Foundation Volatility Framework 2.6.1
*** Failed to import volatility.plugins.registry.shutdown (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.getservicesids (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.timeliner (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.malware.apihooks (NameError: name 'distorm3' is not defined)
*** Failed to import volatility.plugins.malware.servicediff (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.registry.userassist (ImportError: No module named Crypto.Hash)
```

Figure 3: Cloning of the legacy Volatility 2 source tree from GitHub and a first run of `python2 vol.py --info`, which reports several plugin import failures due to missing `Crypto.Hash` and `distorm3` dependencies.

```
*** Failed to import volatility.plugins.registry.shellbags (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.evlogs (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.tcaudit (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.registry.dumpregistry (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.registry.lsadump (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.malware.threads (NameError: name 'distorm3' is not defined)
*** Failed to import volatility.plugins.mac.apihooks_kernel (ImportError: No module named distorm3)
*** Failed to import volatility.plugins.registry.amcache (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.mac.check_syscall_shadow (ImportError: No module named distorm3)
*** Failed to import volatility.plugins.malware.svcscan (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.registry.auditpol (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.ssd (NameError: name 'distorm3' is not defined)
*** Failed to import volatility.plugins.registry.registryapi (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.mac.apihooks (ImportError: No module named distorm3)
*** Failed to import volatility.plugins.envvars (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.registry.shimcache (ImportError: No module named Crypto.Hash)

Plugins
-----
atoms          - Print session and window station atom tables
atomscan       - Pool scanner for atom tables
bigpools       - Dump the big page pools using BigPagePoolScanner
bioskbd        - Reads the keyboard buffer from Real Mode memory
callbacks      - Print system-wide notification routines
clipboard      - Extract the contents of the windows clipboard
```

Figure 4: Continuation of the Volatility 2 plugin import failures, followed by the start of the full plugin listing bundled with the framework.

```

LinuxAMD64PagedMemory      - Linux-specific AMD 64-bit address space.
MachOAddressSpace          - Address space for mach-o files to support atc-ny memory reader
OSXPmemELF                 - This AS supports VirtualBox ELF64 coredump format
QemuCoreDumpElf            - This AS supports Qemu ELF32 and ELF64 coredump format
SkipDuplicatesAMD64PagedMemory - Windows 8/10-specific AMD 64-bit address space.
VMWareAddressSpace         - This AS supports VMWare snapshot (VMSS) and saved state (VMSS) files
VMWareMetaAddressSpace     - This AS supports the VMEM format with VMSN/VMSS metadata
VirtualBoxCoreDumpElf64    - This AS supports VirtualBox ELF64 coredump format
WindowsAMD64PagedMemory    - Windows-specific AMD 64-bit address space.
WindowsCrashDumpSpace32    - This AS supports windows Crash Dump format
WindowsCrashDumpSpace64    - This AS supports windows Crash Dump format
WindowsCrashDumpSpace64BitMap - This AS supports Windows BitMap Crash Dump format
WindowsHiberFileSpace32    - This is a hibernate address space for windows hibernation files.

Scanner Checks
-----
CheckPoolSize              - Check pool block size
CheckPoolType              - Check the pool type
KPCRScannerCheck           - Checks the self referential pointers to find KPCRs
MultiPrefixFinderCheck     - Checks for multiple strings per page, finishing at the offset
MultiStringFinderCheck     - Checks for multiple strings per page
PoolTagCheck               - This scanner checks for the occurrence of a pool tag

(kali@kali)-[~/volatility3/volatility]

```

Figure 5: Further plugin listing, including supported address spaces (Windows, Linux, macOS, VMware, VirtualBox) and the pool-scanner checks used by signature-based plugins.

```

(kali@kali)-[~/volatility3/volatility]
$ wget https://www.dropbox.com/s/x41f2lyhlrixts6/memdumpWin7.mem
--2026-09-14 11:51:30-- https://www.dropbox.com/s/x41f2lyhlrixts6/memdumpWin7.mem
Resolving www.dropbox.com (www.dropbox.com)... 162.125.64.18, 2620:100:6024:18::a27d:4412
Connecting to www.dropbox.com (www.dropbox.com)|162.125.64.18|:443... connected.
HTTP request sent, awaiting response ... 302 Found
Location: https://www.dropbox.com/scl/fi/1y68wpuiq1fsvrpq9egg/memdumpWin7.mem?rlkey=pk3thrk3p2fk9m90mywksx3s1 [
following]
--2026-09-14 11:51:31-- https://www.dropbox.com/scl/fi/1y68wpuiq1fsvrpq9egg/memdumpWin7.mem?rlkey=pk3thrk3p2fk
9m90mywksx3s1
Reusing existing connection to www.dropbox.com:443.
HTTP request sent, awaiting response ... 302 Found
Location: https://uca793e97106dfd7e41aeec42499.dl.dropboxusercontent.com/cd/0/inline/DIE8EiflGVUgb3uW0iIypYK5T0F
-lqWF4jftQcMCKkDY9s-u03C0QtCLK7UThyzHyHIFR5Hv-bYH4u68arx7PzMQzIuzNip0kz6wTt8jt3Egz8_ojt5gKqDrAYBkHosjuR0/file# [
following]
--2026-09-14 11:51:32-- https://uca793e97106dfd7e41aeec42499.dl.dropboxusercontent.com/cd/0/inline/DIE8EiflGVUg
b3uW0iIypYK5T0F-lqWF4jftQcMCKkDY9s-u03C0QtCLK7UThyzHyHIFR5Hv-bYH4u68arx7PzMQzIuzNip0kz6wTt8jt3Egz8_ojt5gKqDrAYBk
HosjuR0/file
Resolving uca793e97106dfd7e41aeec42499.dl.dropboxusercontent.com (uca793e97106dfd7e41aeec42499.dl.dropboxusercon
tent.com)... 162.125.64.15, 2620:100:6024:15::a27d:440f
Connecting to uca793e97106dfd7e41aeec42499.dl.dropboxusercontent.com (uca793e97106dfd7e41aeec42499.dl.dropboxuse
rcontent.com)|162.125.64.15|:443... connected.
HTTP request sent, awaiting response ... 302 Found
Location: /cd/0/inline2/DIE91tb9nuOgCQceI20lwV6lAkGKG7YEfohXMrXCA0o4zeBRP4oLa0FpC6CNfV2vpmHxCNcV8Xu7yJ820yhXAJN
smTxLjUQb8xtyCvM5PRIHAUojRSWcvvk-h0azj3wnlDPRsOFntYdqDdoEBQp16mL0qxhG-a5A3ZjCqGr5bTddB8_naPl97sc5dgIpyccEMTd03GT
TJ6DSNhKvz5xqY00Vgaz3X0q0D_SrTEZSeFI4S0spm1prHCZULCfFEkL9791LNIReddZYei8xJsPCqxI1CfRvN-v0SjdDI8_n04dZnUFj0D9Hwd

```

Figure 6: Acquisition of the evidence file: wget retrieving memdumpWin7.mem from a Dropbox share, following several redirects to the final content host.

```

tent.com)... 162.125.64.15, 2620:100:6024:15::a27d:440f
Connecting to uca03782f8d8dcc01c1ace857cb7.dl.dropboxusercontent.com (uca03782f8d8dcc01c1ace857cb7.dl.dropboxuse
rcontent.com)|162.125.64.15|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: /cd/0/inline2/DINTUuF8KruixuEa24-iYnlb2vhc7kZkfcbF6UX60k1LoNugNFScf4RZlsyAdGGVUrmk0HFXH1Nrr-aK3IVK1JEv
3_DADadTAacBmMv7XXwLmm4jwrDUHQx5Re_umKSapv_UWL7o6f8d3fIz_-tnzNCXdm9TQrTL7_1Zf2nveKI74sNk4MsEC41rth1HVMr1BJRsShLE
m3oIVFNLjdTXAO_j69P_t3a9rHKKIKdYkHR-mLLtkdo3z7W7G7zaHl1pyvWt-Gil6vDg5Tt6yqvYrMjbI1VDnnHDzYxfGTNOUFPME8cl_IE6Y_Jb
SAmtHD-ZZZmAtbXVX3n2k_z-Ptgi7wijXAuvXduCeNJaApt_4cFyBA/file [following]
--2026-09-16 05:00:00-- https://uca03782f8d8dcc01c1ace857cb7.dl.dropboxusercontent.com/cd/0/inline2/DINTUuF8Kru
ixuEa24-iYnlb2vhc7kZkfcbF6UX60k1LoNugNFScf4RZlsyAdGGVUrmk0HFXH1Nrr-aK3IVK1JEv3_DADadTAacBmMv7XXwLmm4jwrDUHQx5Re_
umKSapv_UWL7o6f8d3fIz_-tnzNCXdm9TQrTL7_1Zf2nveKI74sNk4MsEC41rth1HVMr1BJRsShLEm3oIVFNLjdTXAO_j69P_t3a9rHKKIKdYkHR
-mLLtkdo3z7W7G7zaHl1pyvWt-Gil6vDg5Tt6yqvYrMjbI1VDnnHDzYxfGTNOUFPME8cl_IE6Y_JbSAmtHD-ZZZmAtbXVX3n2k_z-Ptgi7wijXAu
vXduCeNJaApt_4cFyBA/file
Reusing existing connection to uca03782f8d8dcc01c1ace857cb7.dl.dropboxusercontent.com:443.
HTTP request sent, awaiting response... 200 OK
Length: 1073676288 (1024M) [application/octet-stream]
Saving to: 'memdumpWin7.mem'

memdumpWin7.mem          100%[=====>] 1024M  2.62MB/s   in 9m 1s

2026-09-16 05:10:04 (1.89 MB/s) - 'memdumpWin7.mem' saved [1073676288/1073676288]

(root@kali)-[/home/kali/volatility3/volatility]

```

Figure 7: Continuation of the download, completing at 1,073,676,288 bytes (1024 MB) and confirming the file was saved successfully.

```

(root@kali)-[/home/kali/volatility3/volatility]
# ls -l
total 1048516
-rw-r--r-- 1 root root 1073676288 Sep 16 05:10 memdumpWin7.mem

```

Figure 8: Directory listing confirming the acquired file `memdumpWin7.mem` is present at its full size of 1,073,676,288 bytes.

5.2 Host Identification (Figures 9–27)

```
(root@kali)-[/home/kali/volatility3]
# python3 vol.py -f /home/kali/volatility3/volatility/memdumpWin7.mem windows.info
Volatility 3 Framework 2.28.2
Progress: 0.12 Reading file http://msdl.microsoft.com/download/symbols/ntkrpamp.pdb/2F6693D273A
Progress: 0.2439C22/ntkrpamp.Reading file http://msdl.microsoft.com/download/symbols/ntkrpamp.pdb/2F6693D273A
Progress: 0.3539C22/ntkrpamp.Reading file http://msdl.microsoft.com/download/symbols/ntkrpamp.pdb/2F6693D273A
Progress: 0.4739C22/ntkrpamp.Reading file http://msdl.microsoft.com/download/symbols/ntkrpamp.pdb/2F6693D273A
Progress: 0.5939C22/ntkrpamp.Reading file http://msdl.microsoft.com/download/symbols/ntkrpamp.pdb/2F6693D273A
Progress: 0.7139C22/ntkrpamp.Reading file http://msdl.microsoft.com/download/symbols/ntkrpamp.pdb/2F6693D273A
Progress: 0.8239C22/ntkrpamp.Reading file http://msdl.microsoft.com/download/symbols/ntkrpamp.pdb/2F6693D273A
Progress: 0.9439C22/ntkrpamp.Reading file http://msdl.microsoft.com/download/symbols/ntkrpamp.pdb/2F6693D273A
Progress: 1.0639C22/ntkrpamp.Reading file http://msdl.microsoft.com/download/symbols/ntkrpamp.pdb/2F6693D273A
Progress: 1.1839C22/ntkrpamp.Reading file http://msdl.microsoft.com/download/symbols/ntkrpamp.pdb/2F6693D273A
Progress: 1.3039C22/ntkrpamp.Reading file http://msdl.microsoft.com/download/symbols/ntkrpamp.pdb/2F6693D273A
Progress: 1.4139C22/ntkrpamp.Reading file http://msdl.microsoft.com/download/symbols/ntkrpamp.pdb/2F6693D273A
Progress: 1.5339C22/ntkrpamp.Reading file http://msdl.microsoft.com/download/symbols/ntkrpamp.pdb/2F6693D273A
Progress: 1.6539C22/ntkrpamp.Reading file http://msdl.microsoft.com/download/symbols/ntkrpamp.pdb/2F6693D273A
Progress: 1.7739C22/ntkrpamp.Reading file http://msdl.microsoft.com/download/symbols/ntkrpamp.pdb/2F6693D273A
Progress: 1.8839C22/ntkrpamp.Reading file http://msdl.microsoft.com/download/symbols/ntkrpamp.pdb/2F6693D273A
Progress: 2.0039C22/ntkrpamp.Reading file http://msdl.microsoft.com/download/symbols/ntkrpamp.pdb/2F6693D273A
Progress: 2.1239C22/ntkrpamp.Reading file http://msdl.microsoft.com/download/symbols/ntkrpamp.pdb/2F6693D273A
Progress: 2.2439C22/ntkrpamp.Reading file http://msdl.microsoft.com/download/symbols/ntkrpamp.pdb/2F6693D273A
Progress: 2.3639C22/ntkrpamp.Reading file http://msdl.microsoft.com/download/symbols/ntkrpamp.pdb/2F6693D273A
Progress: 2.4739C22/ntkrpamp.Reading file http://msdl.microsoft.com/download/symbols/ntkrpamp.pdb/2F6693D273A
Progress: 2.5939C22/ntkrpamp.Reading file http://msdl.microsoft.com/download/symbols/ntkrpamp.pdb/2F6693D273A
Progress: 2.7139C22/ntkrpamp.Reading file http://msdl.microsoft.com/download/symbols/ntkrpamp.pdb/2F6693D273A
```

Figure 9: First Volatility 3 run against the evidence, *windows.info*, which begins by downloading the matching kernel PDB symbols from the Microsoft symbol server.

```
Variable      Value
Kernel Base   0x82847000
DTB           0x185000
Symbols file: ///home/kali/volatility3/volatility3/symbols/windows/ntkrpamp.pdb/2F6693D273A446AA8740983BC3D539C2-2.json.xz
Is64Bit       False
IsPAE         True
layer_name    0 WindowsIntelPAE
memory_layer  1 FileLayer
KdDebuggerDataBlock 0x82972c68
NTBuildLab    7601.18939.x86fre.win7sp1_gdr.15
CSDVersion    1
KdVersionBlock 0x82972c40
Major/Minor   15.7601
MachineType   332
KeNumberProcessors 1
SystemTime    2019-01-06 15:09:06+00:00
NtSystemRoot  C:\Windows
NtProductType NtProductWinNt
NtMajorVersion 6
NtMinorVersion 1
PE MajorOperatingSystemVersion 6
```

Figure 10: *windows.info* output identifying the kernel base, DTB, a 32-bit PAE Windows 7 SP1 build (7601), and a recorded *SystemTime* of 2019-01-06 15:09:06 UTC.

```
PE MajorOperatingSystemVersion 6
PE MinorOperatingSystemVersion 1
PE Machine 332
PE TimeDateStamp Wed Jul 22 16:46:25 2015

(root@kali)-[/home/kali/volatility3]
```

Figure 11: Remaining windows.info fields, including the PE timestamp of 22 July 2015, marking when this Windows 7 build was compiled.

```
INFO : volatility.debug : Determining profile based on KDBG search...
      Suggested Profile(s) : Win7SP1x86_23418, Win7SP0x86, Win7SP1x86_24000, Win7SP1x86
      AS Layer1 : IA32PagedMemoryPae (Kernel AS)
      AS Layer2 : FileAddressSpace (/home/kali/volatility3/volatility/memdumpWin7.mem)
      PAE type : PAE
      DTB : 0x185000L
      KDBG : 0x82972c68L
      Number of Processors : 1
      Image Type (Service Pack) : 1
      KPCR for CPU 0 : 0x82973d00L
      KUSER_SHARED_DATA : 0xffdf0000L
      Image date and time : 2019-01-06 15:09:06 UTC+0000
      Image local date and time : 2019-01-06 07:09:06 -0800

(root@kali)-[/home/kali/volatility]
```

Figure 12: Legacy Volatility 2 imageinfo output on the same file, independently suggesting the Win7SP1x86 profile family and reporting the image date and time in two time zones.

```
Legend: (S) = Stable (V) = Volatile

Registry: \REGISTRY\MACHINE\BCD00000000
Key name: NewStoreRoot (S)
Last updated: 2019-01-06 18:02:44 UTC+0000

Subkeys:
(S) Description
(S) Objects

Values:
Registry: \??\C:\Users\IEUser\ntuser.dat
Key name: CMI-CreateHive{6A1C4018-979D-4291-A7DC-7AED1C75B67C} (S)
Last updated: 2019-01-06 15:02:48 UTC+0000

Subkeys:
(S) AppEvents
(S) Console
(S) Control Panel
(S) Environment
(S) EUDC
(S) Keyboard Layout
(S) Network
(S) Printers
```

Figure 13: printkey output for \REGISTRY\MACHINE\BCD00000000 and the start of the SOFTWARE hive rooted at \Users\IEUser\ntuser.dat, listing standard per-user subkeys.

```

Legend: (S) = Stable   (V) = Volatile

Registry: \??\C:\Users\IEUser\ntuser.dat
Key name: Volatile Environment (V)
Last updated: 2019-01-06 15:02:48 UTC+0000

Subkeys:
(V) 1

Values:
REG_SZ      LOGONSERVER      : (V) \\IE8WIN7
REG_SZ      USERDOMAIN      : (V) IE8WIN7
REG_SZ      USERNAME        : (V) IEUser
REG_SZ      USERPROFILE     : (V) C:\Users\IEUser
REG_SZ      HOMEPATH        : (V) \Users\IEUser
REG_SZ      HOMEDRIVE       : (V) C:
REG_SZ      APPDATA         : (V) C:\Users\IEUser\AppData\Roaming
REG_SZ      LOCALAPPDATA    : (V) C:\Users\IEUser\AppData\Local

(root@kali)-[/home/kali/volatility]
#

```

Figure 14: The Volatile Environment subkey of the IEUser hive, giving the logon server, user domain and username IEUser, and confirming the host name IE8WIN7.

```

Legend: (S) = Stable   (V) = Volatile

Registry: \SystemRoot\System32\Config\SOFTWARE
Key name: ProfileList (S)
Last updated: 2015-09-21 09:50:52 UTC+0000

Subkeys:
(S) S-1-5-18
(S) S-1-5-19
(S) S-1-5-20
(S) S-1-5-21-1716914095-909560446-1177810406-1000
(S) S-1-5-21-1716914095-909560446-1177810406-1002

Values:
REG_EXPAND_SZ ProfilesDirectory : (S) %SystemDrive%\Users
REG_EXPAND_SZ Default           : (S) %SystemDrive%\Users\Default
REG_EXPAND_SZ Public            : (S) %SystemDrive%\Users\Public
REG_EXPAND_SZ ProgramData       : (S) %SystemDrive%\ProgramData

(root@kali)-[/home/kali/volatility]
#

```

Figure 15: *ProfileList* key under the *SOFTWARE* hive, enumerating the well-known SIDs and two custom account SIDs ending in -1000 and -1002.

```

Legend: (S) = Stable   (V) = Volatile

Registry: \SystemRoot\System32\Config\SOFTWARE
Key name: S-1-5-21-1716914095-909560446-1177810406-1002 (S)
Last updated: 2015-09-21 09:51:49 UTC+0000

Subkeys:

Values:
REG_EXPAND_SZ ProfileImagePath : (S) C:\Users\sshd_server
REG_DWORD      Flags           : (S) 0
REG_DWORD      State           : (S) 256
REG_BINARY     Sid              : (S)
0x00000000 01 05 00 00 00 00 00 05 15 00 00 00 af 07 56 66 .....Vf
0x00000010 7e ca 36 36 e6 f5 33 46 ea 03 00 00 ~.66..3F....
REG_DWORD      ProfileLoadTimeLow : (S) 0
REG_DWORD      ProfileLoadTimeHigh : (S) 0
REG_DWORD      RefCount           : (S) 1

(root@kali)-[/home/kali/volatility]

```

Figure 16: *Detail of SID ...-1002: ProfileImagePath* resolves to *C:\Users\sshd_server*, an account name not part of a standard Windows 7 installation.

```

Legend: (S) = Stable   (V) = Volatile

Registry: \SystemRoot\System32\Config\SOFTWARE
Key name: S-1-5-21-1716914095-909560446-1177810406-1000 (S)
Last updated: 2019-01-06 15:07:15 UTC+0000

Subkeys:

Values:
REG_EXPAND_SZ ProfileImagePath : (S) C:\Users\IEUser
REG_DWORD      Flags           : (S) 0
REG_DWORD      State           : (S) 0
REG_BINARY     Sid              : (S)
0x00000000 01 05 00 00 00 00 00 05 15 00 00 00 af 07 56 66 .....Vf
0x00000010 7e ca 36 36 e6 f5 33 46 e8 03 00 00 ~.66..3F....
REG_DWORD      ProfileLoadTimeLow : (S) 0
REG_DWORD      ProfileLoadTimeHigh : (S) 0
REG_DWORD      RefCount           : (S) 3
REG_DWORD      RunLogonScriptSync : (S) 0

(root@kali)-[/home/kali/volatility]

```

Figure 17: *Detail of SID ...-1000: ProfileImagePath* resolves to *C:\Users\IEUser*, the primary interactive account, with a *RefCount* of 3 and a load time of 2019-01-06 15:07:15.

```

Registry: \SystemRoot\System32\Config\SOFTWARE
Key name: Winlogon (S)
Last updated: 2019-01-06 15:02:47 UTC+0000

Subkeys:
(S) GPEExtensions
(V) AutoLogonChecked

Values:
REG_SZ      ReportBootOk      : (S) 1
REG_SZ      Shell              : (S) explorer.exe
REG_SZ      PreCreateKnownFolders : (S) {A520A1A4-1780-4FF6-BD18-167343C5AF16}
REG_SZ      Userinit           : (S) C:\Windows\system32\userinit.exe,
REG_SZ      VMApplet            : (S) SystemPropertiesPerformance.exe /pagefile
REG_SZ      WinStationsDisabled : (S) 0
REG_DWORD   DisableCAD         : (S) 1
REG_SZ      scremoveoption      : (S) 0
REG_DWORD   ShutdownFlags      : (S) 2147483687
REG_SZ      DefaultDomainName   : (S)
REG_SZ      DefaultUserName      : (S) IEUser
REG_SZ      AutoAdminLogon       : (S) 1

(root@kali)-[/home/kali/volatility]
#

```

Figure 18: Winlogon key showing AutoAdminLogon enabled with DefaultUserName IEUser, and the default Explorer shell configuration.

Virtual	Physical	Name
0x87c10370	0x27fd5370	[no name]
0x87c1c008	0x27fa3008	\REGISTRY\MACHINE\SYSTEM
0x87c459c8	0x27d8e9c8	\REGISTRY\MACHINE\HARDWARE
0x889c0430	0x1285a430	\\?\C:\Windows\ServiceProfiles\NetworkService\NTUSER.DAT
0x8b46d008	0x16d8c008	\\?\C:\Windows\ServiceProfiles\LocalService\NTUSER.DAT
0x8b5d39b0	0x120049b0	\\?\C:\Users\IEUser\ntuser.dat
0x97104008	0x1c45f008	\SystemRoot\System32\Config\SECURITY
0x9711d9c8	0x1e6a09c8	\SystemRoot\System32\Config\SOFTWARE
0x981f9008	0x1b446008	\SystemRoot\System32\Config\DEFAULT
0x98209518	0x1f3a7518	\SystemRoot\System32\Config\SAM
0x982d55c0	0x1e17c5c0	\REGISTRY\MACHINE\BCD00000000
0xa620b008	0x1997b008	\\?\C:\Users\IEUser\AppData\Local\Microsoft\Windows\UsrClass.dat
0xa65269c8	0x17ad09c8	\\?\C:\Users\sshd_server\ntuser.dat
0xa6539260	0x0f690260	\\?\C:\Users\sshd_server\AppData\Local\Microsoft\Windows\UsrClass.dat
0xa7d2a9c8	0x01c469c8	\\?\C:\System Volume Information\Syscache.hve

```

(root@kali)-[/home/kali/volatility]
#

```

Figure 19: hivelist output enumerating every registry hive loaded in memory, including separate ntuser.dat and UsrClass.dat hives for both IEUser and sshd_server.

```

Legend: (S) = Stable   (V) = Volatile

Registry: \REGISTRY\MACHINE\HARDWARE
Key name: HARDWARE (S)
Last updated: 2019-01-06 18:02:40 UTC+0000

Subkeys:
  (S) ACPI
  (S) DESCRIPTION
  (S) DEVICEMAP

Values:

(root@kali)-[/home/kali/volatility]
#

```

Figure 20: *HARDWARE* hive root, listing the *ACPI*, *DESCRIPTION* and *DEVICEMAP* subkeys used to identify the underlying physical or virtual hardware.

```

Legend: (S) = Stable   (V) = Volatile

Registry: \REGISTRY\MACHINE\HARDWARE
Key name: 0 (S)
Last updated: 2019-01-06 18:02:40 UTC+0000

Subkeys:

Values:
REG_BINARY    Component Information : (S)
0x00000000    00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
REG_SZ        Identifier       : (S) x86 Family 6 Model 158 Stepping 9
REG_FULL_RESOURCE_DESCRIPTOR Configuration Data : (S) ♦♦♦♦♦♦♦♦
REG_SZ        ProcessorNameString : (S) Intel(R) Core(TM) i7-7700 CPU @ 3.60GHz
REG_SZ        VendorIdentifier  : (S) GenuineIntel
REG_DWORD     FeatureSet       : (S) 2697805823
REG_DWORD     ~MHz             : (S) 3600
REG_DWORD     Update Status    : (S) 2
REG_BINARY    Update Signature : (S)
0x00000000    00 00 00 00 00 00 00 00 .....
REG_BINARY    Previous Update Signature : (S)
0x00000000    00 00 00 00 00 00 00 00 .....
REG_DWORD     Platform ID      : (S) 2

(root@kali)-[/home/kali/volatility]

```

Figure 21: *DESCRIPTION\System\0* subkey identifying the virtual CPU as an Intel Core i7-7700 @ 3.60GHz, the specification of the physical host beneath the hypervisor.

```

Key name: System (S)
Last updated: 2019-01-06 18:02:44 UTC+0000

Subkeys:
  (S) CentralProcessor
  (S) FloatingPointProcessor
  (S) MultifunctionAdapter

Values:
REG_BINARY    Component Information : (S)
0x00000000 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
REG_SZ        Identifier : (S) AT/AT COMPATIBLE
REG_FULL_RESOURCE_DESCRIPTOR Configuration Data : (S) ♦♦♦♦♦♦♦♦
♦♦?♦

REG_SZ        SystemBiosDate : (S) 06/23/99
REG_MULTI_SZ  SystemBiosVersion : (S) ['VBOX - 1', '', '']
REG_DWORD     BootArchitecture : (S) 3
REG_DWORD     PreferredProfile : (S) 0
REG_DWORD     Capabilities : (S) 1345
REG_MULTI_SZ  VideoBiosVersion : (S) ['Oracle VM VirtualBox Version 6.0.0 VGA BIOS', 'Oracle VM VirtualBox Versi
on 6.0.0 VGA BIOS', 'Oracle VM VirtualBox Version 6.0.0', 'Oracle VM VirtualBox Version 6.0.0', '', '', '', '',
'', '', '', '', '', '', '', '', '']

(root@kali)-[/home/kali/volatility]

```

Figure 22: Continuation of the System hardware key, with SystemBiosVersion and VideoBiosVersion strings identifying Oracle VM VirtualBox 6.0.0 as the virtualisation platform.

```

Legend: (S) = Stable (V) = Volatile

Registry: \REGISTRY\MACHINE\SYSTEM
Key name: CMI-CreateHive{F10156BE-0E87-4EFB-969E-5DA29D131144} (S)
Last updated: 2019-01-06 18:02:40 UTC+0000

Subkeys:
  (S) ControlSet001
  (S) ControlSet002
  (S) MountedDevices
  (S) RNG
  (S) Select
  (S) Setup
  (S) WPA
  (V) CurrentControlSet

Values:

(root@kali)-[/home/kali/volatility]

```

Figure 23: SYSTEM hive root, showing ControlSet001, ControlSet002 and the CurrentControlSet symbolic link used to resolve the active configuration.

```
Legend: (S) = Stable   (V) = Volatile

Registry: \REGISTRY\MACHINE\SYSTEM
Key name: Enum (S)
Last updated: 2019-01-06 15:03:07 UTC+0000

Subkeys:
(S) ACPI
(S) DISPLAY
(S) HTREE
(S) IDE
(S) LPTENUM
(S) PCI
(S) PCIIDE
(S) Root
(S) SCSI
(S) STORAGE
(S) SW
(S) USBSTOR
(S) VMBUS
(S) WpdBusEnumRoot
```

Figure 24: *CurrentControlSet\Enum* key, enumerating device classes recognised on the system, including *USBSTOR* and *VMBUS*.

```

Legend: (S) = Stable   (V) = Volatile

Registry: \REGISTRY\MACHINE\SYSTEM
Key name: PCI (S)
Last updated: 2019-01-06 18:02:40 UTC+0000

Subkeys:
(S) VEN_1000&DEV_0054&SUBSYS_1F091028&REV_01
(S) VEN_1002&DEV_515E&SUBSYS_01E61028&REV_02
(S) VEN_1022&DEV_2000&SUBSYS_20001022&REV_40
(S) VEN_106B&DEV_003F&SUBSYS_00000000&REV_00
(S) VEN_14E4&DEV_1659&SUBSYS_01E61028&REV_11
(S) VEN_15AD&DEV_0790&SUBSYS_079015AD&REV_02
(S) VEN_15AD&DEV_07A0&SUBSYS_07A015AD&REV_01
(S) VEN_8086&DEV_032C&SUBSYS_00000000&REV_09
(S) VEN_8086&DEV_100E&SUBSYS_001E8086&REV_02
(S) VEN_8086&DEV_1237&SUBSYS_00000000&REV_02
(S) VEN_8086&DEV_265C&SUBSYS_00000000&REV_00
(S) VEN_8086&DEV_2778&SUBSYS_01E61028&REV_00
(S) VEN_8086&DEV_2779&SUBSYS_01E61028&REV_00
(S) VEN_8086&DEV_27C8&SUBSYS_01E61028&REV_01
(S) VEN_8086&DEV_27C9&SUBSYS_01E61028&REV_01
(S) VEN_8086&DEV_27CA&SUBSYS_01E61028&REV_01
(S) VEN_8086&DEV_27CC&SUBSYS_01E61028&REV_01

```

Figure 25: *CurrentControlSet\Enum\PCI* key, listing the virtual PCI devices exposed by VirtualBox to the guest, identified by vendor and device IDs.

```

Legend: (S) = Stable   (V) = Volatile

Registry: \REGISTRY\MACHINE\SYSTEM
Key name: CurrentControlSet (V)
Last updated: 2019-01-06 18:02:40 UTC+0000

Subkeys:

Values:
REG_LINK      SymbolicLinkValue : (V) \Registry\Machine\System\ControlSet001

(root@kali)-[/home/kali/volatility]
#

```

Figure 26: *CurrentControlSet* symbolic link confirmed to point at *ControlSet001*, meaning *ControlSet001* was active at the time of capture.

```
Legend: (S) = Stable   (V) = Volatile

Registry: \REGISTRY\MACHINE\SYSTEM
Key name: ComputerName (S)
Last updated: 2015-09-21 09:48:51 UTC+0000

Subkeys:

Values:
REG_SZ          : (S) mnmsrvc
REG_SZ          : (S) IE8WIN7
ComputerName    : (S) IE8WIN7

(root@kali)-[/home/kali/volatility]
#
```

Figure 27: *ComputerName* key confirming the NetBIOS host name *IE8WIN7* for the machine under examination.

5.3 Network, USB and Account Evidence (Figures 28–48)

```
(root@kali)-[/home/kali/volatility]
# python2 vol.py -f /home/kali/volatility3/volatility/memdumpWin7.mem --profile=Win7SP1x86_23418 netscan | gre
p TCPv4

Volatility Foundation Volatility Framework 2.6.1
0x21478bb0 TCPv4 0.0.0.0:49155 0.0.0.0:0 LISTENING 432 servic
es.exe
0x269ff708 TCPv4 0.0.0.0:49156 0.0.0.0:0 LISTENING 440 lsass.
exe
0x3e446008 TCPv4 0.0.0.0:49154 0.0.0.0:0 LISTENING 880 svchos
t.exe
0x3e446d98 TCPv4 0.0.0.0:49154 0.0.0.0:0 LISTENING 880 svchos
t.exe
0x3e46e0c0 TCPv4 192.168.56.8:139 0.0.0.0:0 LISTENING 4 System
0x3e671988 TCPv4 0.0.0.0:445 0.0.0.0:0 LISTENING 4 System
0x3e72e530 TCPv4 0.0.0.0:135 0.0.0.0:0 LISTENING 672 svchos
t.exe
0x3e72ee48 TCPv4 0.0.0.0:135 0.0.0.0:0 LISTENING 672 svchos
t.exe
0x3e736388 TCPv4 0.0.0.0:49152 0.0.0.0:0 LISTENING 340 winini
t.exe
0x3e736d98 TCPv4 0.0.0.0:49152 0.0.0.0:0 LISTENING 340 winini
t.exe
0x3e79d528 TCPv4 0.0.0.0:49153 0.0.0.0:0 LISTENING 724 svchos
```

Figure 28: Volatility 2 netscan output listing TCP listening sockets, including standard Windows services and a socket bound to 192.168.56.8:139.

```
Offset(V)  Name          PID  PPID  Thds  Hnds  Sess  Wow64  Start          Exit
-----
0x8413a908 System        4    0    73   518   -----  0  2019-01-06 18:02:44 UTC+0000

(root@kali)-[/home/kali/volatility]
#
```

Figure 29: pstree/pslist entry for the System process (PID 4), the root of the Windows process tree, started at the memory image's boot time.

```

Usage: Volatility - A memory forensics analysis platform.

Options:
  -h, --help                list all available options and their default values.
                           Default values may be set in the configuration file
                           (/etc/volatilityrc)
  --conf-file=/root/.volatilityrc
                           User based configuration file
  -d, --debug               Debug volatility
  --plugins=PLUGINS        Additional plugin directories to use (colon separated)
  --info                   Print information about all registered objects
  --cache-directory=/root/.cache/volatility
                           Directory where cache files are stored
  --cache                  Use caching
  --tz=TZ                  Sets the (Olson) timezone for displaying timestamps
                           using pytz (if installed) or tzset
  -f FILENAME, --filename=FILENAME
                           Filename to use when opening an image
  --profile=WinXPSP2x86    Name of the profile to load (use --info to see a list
                           of supported profiles)
  -l LOCATION, --location=LOCATION
                           A URN location from which to load an address space
  -w, --write              Enable write support
  --dtb=DTB               DTB Address
  --shift=SHIFT            Mac KASLR shift address

```

Figure 30: General Volatility 2 usage help, listing global options such as `--profile`, `--filename` and `--dtb` used when invoking any plugin.

```

(root@kali)-[/home/kali/volatility]
# strings /home/kali/volatility3/volatility/memdumpWin7.mem | grep "192.168.56.5" > process_1.txt

(root@kali)-[/home/kali/volatility]
# ls -l process_1.txt

-rw-r--r-- 1 root root 13858 Sep 16 06:19 process_1.txt

(root@kali)-[/home/kali/volatility]
#

```

Figure 31: A raw string search of the memory image for the internal address 192.168.56.5, redirected to `process_1.txt` for review.

```

(root@kali)-[/home/kali/volatility]
# xxd process_1.txt | head -n 30
00000000: 2f73 6563 7265 745f 6669 6c65 2e64 6f63 /secret_file.doc
00000010: 7868 7474 7073 3a2f 2f77 7777 2e67 6f6f xhttps://www.goo
00000020: 676c 652e 636f 6d2f 7365 6172 6368 3f71 gle.com/search?q
00000030: 3d31 3932 2e31 3638 2e35 362e 3525 3246 =192.168.56.5%2F
00000040: 7365 6372 6574 5f66 696c 652e 646f 6378 secret_file.docx
00000050: 266f 713d 3139 322e 3136 382e 3536 2e35 &oq=192.168.56.5
00000060: 2532 4673 6563 7265 745f 6669 6c65 2e64 %2Fsecret_file.d
00000070: 6f63 7826 6171 733d 6368 726f 6d65 2e2e ocx&aqs=chrome..
00000080: 3639 6935 386a 3639 6935 3726 736f 7572 69i58j69i57&sour
00000090: 6365 6964 3d63 6872 6f6d 6526 6965 3d55 ceid=chrome&ie=U
000000a0: 5446 2d38 0a74 7470 3a2f 2f31 3932 2e31 TF-8.ttp://192.1
000000b0: 3638 2e35 362e 352f 7365 6372 6574 5f66 68.56.5/secret_f
000000c0: 696c 652e 646f 6378 0a68 7474 703a 2f2f ile.docx.http://
000000d0: 3139 322e 3136 382e 3536 2e35 2f73 6563 192.168.56.5/sec
000000e0: 7265 745f 6669 6c65 2e64 6f63 780a 7470 ret_file.docx.tp
000000f0: 3a2f 2f31 3932 2e31 3638 2e35 362e 352f ://192.168.56.5/
00000100: 0a09 6565 3462 6365 3530 2d31 6131 632d ..ee4bce50-1a1c-
00000110: 3439 6166 2d38 3165 372d 6438 6137 3138 49af-81e7-d8a718
00000120: 3037 3435 3836 3139 322e 3136 382e 3536 074586192.168.56

```

Figure 32: Hex/ASCII dump of the extracted strings, revealing browser artefacts referencing `/secret_file.docx`, a Google search for the same path, and the URL `http://192.168.56.5/secret_file.docx`.

```

*****
CommandProcess: conhost.exe Pid: 2000
CommandHistory: 0x320960 Application: sshd.exe Flags: Allocated
CommandCount: 0 LastAdded: -1 LastDisplayed: -1
FirstCommand: 0 CommandCountMax: 50
ProcessHandle: 0x54
Cmd #29 @ 0x3200c4: 3?4?2???2
Cmd #37 @ 0x3200c4: 3?4?2???2
*****
CommandProcess: conhost.exe Pid: 3988
CommandHistory: 0x110448 Application: cmd.exe Flags: Allocated, Reset
CommandCount: 3 LastAdded: 2 LastDisplayed: 2
FirstCommand: 0 CommandCountMax: 50
ProcessHandle: 0x5c
Cmd #0 @ 0x10dc40: ipconfig
Cmd #1 @ 0x107f90: cd Downloads
Cmd #2 @ 0x1147a0: copy secret_file.docx F:
Cmd #22 @ 0xff818488: ?
Cmd #25 @ 0xff818488: ?
Cmd #36 @ 0xe00c4: ?????
Cmd #37 @ 0x10cff0: ?????

```

```

(root@kali)-[/home/kali/volatility]
# █

```

Figure 33: *cmdscan* output for two *conhost.exe* sessions: one attached to *sshd.exe*, and a second attached to *cmd.exe* showing three recorded commands, including *copy secret_file.docx F:*.

Offset(V)	Name	PID	PPID	Thds	Hnds	Sess	Wow64	Start	Exit
0x842ddd28	conhost.exe	3988	348	2	52	1	0	2019-01-06 15:06:11 UTC+0000	

(root@kali)-[/home/kali/volatility]

Figure 34: *pslist* entry for *conhost.exe*, PID 3988, parent 348, started 2019-01-06 15:06:11, the console host attached to the interactive *cmd.exe* session.

```
*****
ConsoleProcess: conhost.exe Pid: 2000
Console: 0xe181c0 CommandHistorySize: 50
HistoryBufferCount: 2 HistoryBufferMax: 4
OriginalTitle: C:\Program Files\OpenSSH\bin\cygrunsrv.exe
Title: C:\Program Files\OpenSSH\bin\cygrunsrv.exe
AttachedProcess: sshd.exe Pid: 2016 Handle: 0x54
-----
CommandHistory: 0x320960 Application: sshd.exe Flags: Allocated
CommandCount: 0 LastAdded: -1 LastDisplayed: -1
FirstCommand: 0 CommandCountMax: 50
ProcessHandle: 0x54
-----
CommandHistory: 0x3207f0 Application: - Flags:
CommandCount: 0 LastAdded: -1 LastDisplayed: -1
FirstCommand: 0 CommandCountMax: 50
ProcessHandle: 0x0
-----
Screen 0x3360a0 X:80 Y:300
Dump:
*****
ConsoleProcess: conhost.exe Pid: 3988
Console: 0xe181c0 CommandHistorySize: 50
HistoryBufferCount: 2 HistoryBufferMax: 4
```

Figure 35: *consoles* output for the first *conhost.exe* (PID 2000), whose window title identifies it as hosting *C:\Program Files\OpenSSH\bin\cygrunsrv.exe* with *sshd.exe* (PID 2016) attached.

```

OriginalTitle: %SystemRoot%\system32\cmd.exe
Title: C:\Windows\system32\cmd.exe
AttachedProcess: cmd.exe Pid: 3996 Handle: 0x5c
-----
CommandHistory: 0x1105d8 Application: ipconfig.exe Flags:
CommandCount: 0 LastAdded: -1 LastDisplayed: -1
FirstCommand: 0 CommandCountMax: 50
ProcessHandle: 0x0
-----
CommandHistory: 0x110448 Application: cmd.exe Flags: Allocated, Reset
CommandCount: 3 LastAdded: 2 LastDisplayed: 2
FirstCommand: 0 CommandCountMax: 50
ProcessHandle: 0x5c
Cmd #0 at 0x10dc40: ipconfig
Cmd #1 at 0x107f90: cd Downloads
Cmd #2 at 0x1147a0: copy secret_file.docx F:
-----
Screen 0xf63b8 X:80 Y:300
Dump:
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\IEUser>ipconfig

```

Figure 36: consoles output for the second conhost.exe (PID 3988), confirming a cmd.exe session and the recorded command history: ipconfig, cd Downloads, copy secret_file.docx F:.

```

Windows IP Configuration

Ethernet adapter Local Area Connection 3:

    Connection-specific DNS Suffix  . : 
    Link-local IPv6 Address . . . . . : fe80::1571:7b34:b2b1:bbe4%18
    IPv4 Address. . . . . : 192.168.56.8
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 

Tunnel adapter isatap.{7FD51246-C28D-46B6-AA98-EB2C150C388B}:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . : 

Tunnel adapter Teredo Tunneling Pseudo-Interface:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . : 

C:\Users\IEUser>cd Downloads

C:\Users\IEUser\Downloads>copy secret_file.docx F:

```

Figure 37: Screen buffer replay of the `cmd.exe` session, showing the `ipconfig` output (host address 192.168.56.8) followed by the operator navigating to Downloads and issuing the copy command.

```
1 file(s) copied.  
C:\Users\IEUser\Downloads>  
(root@kali)-[/home/kali/volatility]  
#
```

Figure 38: Continuation of the screen buffer replay, confirming the shell's own "1 file(s) copied" response to the `copy secret_file.docx F:` command.

```
Legend: (S) = Stable (V) = Volatile  
  
Registry: \REGISTRY\MACHINE\SYSTEM  
Key name: DeviceClasses (S)  
Last updated: 2019-01-06 15:03:08 UTC+0000  
  
Subkeys:  
(S) {0e0b6031-5213-4934-818b-38d90ced39db}  
(S) {10497b1b-ba51-44e5-8318-a65c837b6661}  
(S) {2accfe60-c130-11d2-b082-00a0c91efb8b}  
(S) {2e34d650-5819-42ca-84ae-d30803bae505}  
(S) {32412632-86cb-44a2-9b5c-50d1417354f5}  
(S) {3375baf4-9e15-4b30-b765-67acb10d607b}  
(S) {34d14be3-dee4-41c8-9ae7-6b174977c192}  
(S) {35fa2e29-ea23-4236-96ae-3a6ebacba440}  
(S) {3abf6f2d-71c4-462a-8a92-1e6861e6af27}  
(S) {4116f60b-25b3-4662-b732-99a6111edc0b}  
(S) {4afa3d53-74a7-11d0-be5e-00a0c9062857}  
(S) {4d1e55b2-f16f-11cf-88cb-001111000030}  
(S) {4d36e978-e325-11ce-bfc1-08002be10318}  
(S) {53f56307-b6bf-11d0-94f2-00a0c91efb8b}  
(S) {53f56308-b6bf-11d0-94f2-00a0c91efb8b}  
(S) {53f5630d-b6bf-11d0-94f2-00a0c91efb8b}  
(S) {53f5630e-b6bf-11d0-94f2-00a0c91efb8b}  
(S) {53f56311-b6bf-11d0-94f2-00a0c91efb8b}
```

Figure 39: `DeviceClasses` key under the `SYSTEM` hive, enumerating GUID-named device class registrations used to identify attached hardware, including storage volumes.

```
(S) {57164f39-9115-4e78-ab55-382f3bd5422d}
(S) {5b45201d-f2f2-4f3b-85bb-30ff1f953599}
(S) {6ac27878-a6fa-4155-ba85-f98f491d4f33}
(S) {72631e54-78a4-11d0-bcf7-00aa00b7b32a}
(S) {866519b5-3f07-4c97-b7df-24c5d8a8ccb8}
(S) {9527e630-d0ae-497b-adce-e80ab0175caf}
(S) {97f76ef0-f883-11d0-af1f-0000f800845c}
(S) {a9a0f4e7-5a45-4d96-b827-8a841e8c03e6}
(S) {e849804e-c719-43d8-ac88-96b894c191e2}
(S) {f33fdc04-d1ac-4e8e-9a30-19bbd4b108ae}

Values:

(root@kali)-[/home/kali/volatility]
```

Figure 40: Continuation of the DeviceClasses GUID listing.

```
Legend: (S) = Stable (V) = Volatile

Registry: \REGISTRY\MACHINE\SYSTEM
Key name: {53f56307-b6bf-11d0-94f2-00a0c91efb8b} (S)
Last updated: 2019-01-06 15:02:54 UTC+0000

Subkeys:
(S) ##?#IDE#DiskVBOX_HARDDISK_____1.0_____#56106af1716061.0.0#{53f56307-b6bf-11d0-94f2-00a0c91efb8b}
(S) ##?#IDE#DiskVirtual_HD_____1.1.0_____#5635dc70406060.0.0#{53f56307-b6bf-11d0-94f2-00a0c91efb8b}
(S) ##?#IDE#DiskVMware_Virtual_IDE_Hard_Drive_____00000001#562eba496060.0.0#{53f56307-b6bf-11d0-94f2-00a0c91efb8b}
(S) ##?#SCSI#Disk&Ven_Dell&Prod_VIRTUAL_DISK#6617b134376060000000#{53f56307-b6bf-11d0-94f2-00a0c91efb8b}
(S) ##?#USBSTOR#Disk&Ven_General&Prod_UDisk&Rev_5.00#661bec0f48606_60#{53f56307-b6bf-11d0-94f2-00a0c91efb8b}

Values:

(root@kali)-[/home/kali/volatility]
```

Figure 41: Enum\{53f56307-...} disk class key, listing every disk device the system has seen, including two virtual hard disks and a USBSTOR device described as Ven_General Prod_UDisk Rev_5.00.

```

Legend: (S) = Stable (V) = Volatile

Registry: \REGISTRY\MACHINE\SYSTEM
Key name: ##?#USBSTOR#Disk&Ven_General&Prod_UDisk&Rev_5.00#681bec0f48606_60#{53f56307-b6bf-11d0-94f2-00a0c91efb8b} (S)
Last updated: 2019-01-06 15:02:54 UTC+0000

Subkeys:
(S) #
(V) Control

Values:
REG_SZ DeviceInstance : (S) USBSTOR\Disk&Ven_General&Prod_UDisk&Rev_5.00\681bec0f48606_60

(root@kali)-[/home/kali/volatility]
#

```

Figure 42: Detail of the USBSTOR device instance path for the General UDisk, confirming its presence under the disk device class.

```

Legend: (S) = Stable (V) = Volatile

Registry: \REGISTRY\MACHINE\SYSTEM
Key name: ##?#USBSTOR#Disk&Ven_General&Prod_UDisk&Rev_5.00#681bec0f48606_60#{53f56307-b6bf-11d0-94f2-00a0c91efb8b} (S)
Last updated: 2019-01-06 15:02:54 UTC+0000

Subkeys:
(S) #
(V) Control

Values:
REG_SZ DeviceInstance : (S) USBSTOR\Disk&Ven_General&Prod_UDisk&Rev_5.00\681bec0f48606_60

(root@kali)-[/home/kali/volatility]
#

```

Figure 43: Repeat view of the same USBSTOR device instance key, confirming its DeviceInstance string.

```

Legend: (S) = Stable (V) = Volatile

Registry: \REGISTRY\MACHINE\SYSTEM
Key name: USBSTOR (S)
Last updated: 2019-01-06 15:02:54 UTC+0000

Subkeys:
(S) Disk&Ven_General&Prod_UDisk&Rev_5.00

Values:

(root@kali)-[/home/kali/volatility]
#

```

Figure 44: *USBSTOR root key, showing a single subkey for Disk&Ven_General&Prod_UDisk&Rev_5.00 — the only USB mass-storage device ever connected to this system image.*

```

Legend: (S) = Stable (V) = Volatile

Registry: \REGISTRY\MACHINE\SYSTEM
Key name: Disk&Ven_General&Prod_UDisk&Rev_5.00 (S)
Last updated: 2019-01-06 15:02:54 UTC+0000

Subkeys:
  (S) 661bec0f48608_80

Values:

(root@kali)-[/home/kali/volatility]
#

```

Figure 45: *Serial-number subkey of the USB device, dated 2019-01-06 15:02:54, matching the timeframe of the incident.*

```

Registry: \REGISTRY\MACHINE\SYSTEM
Key name: 661bec0f48608_80 (S)
Last updated: 2019-01-06 15:02:54 UTC+0000

Subkeys:
  (S) Device Parameters
  (S) LogConf
  (S) Properties
  (V) Control

Values:
REG_SZ DeviceDesc : (S) @disk.inf,%disk_devdesc%;Disk drive
REG_DWORD Capabilities : (S) 0
REG_MULTI_SZ HardwareID : (S) ['USBSTOR\\DiskGeneral_UDisk____5.00', 'USBSTOR\\DiskGeneral_UDisk____', 'USBSTOR\\DiskGeneral_', 'USBSTOR\\General_UDisk____5', 'General_UDisk____5', 'USBSTOR\\GenDisk', 'GenDisk', '', '']
REG_MULTI_SZ CompatibleIDs : (S) ['USBSTOR\\Disk', 'USBSTOR\\RAW', '', '']
REG_SZ ContainerID : (S) {421e7e37-11dd-11e9-b3cf-806e6f6e6963}
REG_DWORD ConfigFlags : (S) 0
REG_SZ ClassGUID : (S) {4d36e967-e325-11ce-bfc1-08002be10318}
REG_SZ Class : (S) DiskDrive
REG_SZ Mfg : (S) @disk.inf,%genmanufacturer%;(Standard disk drives)
REG_SZ Service : (S) disk
REG_SZ FriendlyName : (S) General UDisk USB Device

```

Figure 46: *Device Parameters and Properties of the USB disk, including its HardwareID and CompatibleIDs strings identifying it as a generic USB mass-storage disk drive.*

5.4 Browser History, Shellbags and Password Recovery (Figures 49–61)

```
*****
Process: 2524 explorer.exe
Cache type "URL " at 0x3156080
Record length: 0x200
Location: https://www.google.com/chrome/static/images/favicons/favicon.ico
Last modified: 2018-04-26 05:30:00 UTC+0000
Last accessed: 2018-09-17 14:54:34 UTC+0000
File Offset: 0x200, Data Offset: 0xac, Data Length: 0xbc
File: favicon[2].ico
Data: HTTP/1.1 200 OK
Content-Type: image/x-icon
X-Content-Type-Options: nosniff
X-XSS-Protection: 1; mode=block
Alt-Svc: quic=":443"; ma=2592000; v="44,43,39,35"
Transfer-Encoding: chunked

~U:ieuser

*****
Process: 2524 explorer.exe
Cache type "URL " at 0x3156480
Record length: 0x180
Location: https://www.jam-software.de/css/reset_v4.css
Last modified: 2015-09-11 10:49:57 UTC+0000
```

Figure 49: *iehistory* output listing cached URL records from *explorer.exe* (PID 2524), showing ordinary browsing artefacts such as favicon and stylesheet requests.

```
(root@kali)-[/home/kali/volatility]
# python2 vol.py -f /home/kali/volatility3/volatility/memdumpWin7.mem --profile=Win7SP1x86_23418 iehistory | g
rep 2019
Volatility Foundation Volatility Framework 2.6.1

(root@kali)-[/home/kali/volatility]
#
```

Figure 50: A targeted *iehistory* query filtered for the string "2019", returning no matching Internet Explorer history records for that year.

```
(root@kali)-[/home/kali/volatility]
# python2 vol.py -f /home/kali/volatility3/volatility/memdumpWin7.mem --profile=Win7SP1x86_23418 shellbags | g
rep "Last updated: 2019-01-06" | sort
Volatility Foundation Volatility Framework 2.6.1
Last updated: 2019-01-06 14:32:21 UTC+0000
Last updated: 2019-01-06 14:32:53 UTC+0000
Last updated: 2019-01-06 14:55:45 UTC+0000
Last updated: 2019-01-06 15:04:40 UTC+0000
Last updated: 2019-01-06 15:04:40 UTC+0000
Last updated: 2019-01-06 15:04:40 UTC+0000
Last updated: 2019-01-06 15:07:03 UTC+0000

(root@kali)-[/home/kali/volatility]
#
```

Figure 51: *shellbags* output filtered and sorted for entries updated on 2019-01-06, producing a timeline of folder-view accesses clustered around 14:32 to 15:07 UTC.

```

(root@kali)-[/home/kali/volatility]
# python2 vol.py -f /home/kali/volatility3/volatility/memdumpWin7.mem --profile=Win7SP1x86_23418 shellbags --o
utput=html --output-file=shellbags.html

Volatility Foundation Volatility Framework 2.6.1
*** Failed to import volatility.plugins.malware.apihooks (NameError: name 'distorm3' is not defined)
*** Failed to import volatility.plugins.malware.threads (NameError: name 'distorm3' is not defined)
*** Failed to import volatility.plugins.mac.apihooks_kernel (ImportError: No module named distorm3)
*** Failed to import volatility.plugins.mac.check_syscall_shadow (ImportError: No module named distorm3)
*** Failed to import volatility.plugins.ssdts (NameError: name 'distorm3' is not defined)
*** Failed to import volatility.plugins.mac.apihooks (ImportError: No module named distorm3)
Scanning for registries....
Gathering shellbag items and building path tree...
Outputting to: shellbags.html

(root@kali)-[/home/kali/volatility]
# ls shellbags.html

shellbags.html

(root@kali)-[/home/kali/volatility]
#

```

Figure 52: *Generation of an HTML shellbags report (shellbags.html) for offline review, noting the same distorm3-related plugin import failures seen previously.*

```

(root@kali)-[/home/kali/volatility]
# python2 vol.py -f /home/kali/volatility3/volatility/memdumpWin7.mem --profile=Win7SP1x86_23418 printkey -K "
ControlSet001\Enum\USBSTOR"

Volatility Foundation Volatility Framework 2.6.1
*** Failed to import volatility.plugins.malware.apihooks (NameError: name 'distorm3' is not defined)
*** Failed to import volatility.plugins.malware.threads (NameError: name 'distorm3' is not defined)
*** Failed to import volatility.plugins.mac.apihooks_kernel (ImportError: No module named distorm3)
*** Failed to import volatility.plugins.mac.check_syscall_shadow (ImportError: No module named distorm3)
*** Failed to import volatility.plugins.ssdts (NameError: name 'distorm3' is not defined)
*** Failed to import volatility.plugins.mac.apihooks (ImportError: No module named distorm3)
Legend: (S) = Stable (V) = Volatile

Registry: \REGISTRY\MACHINE\SYSTEM
Key name: USBSTOR (S)
Last updated: 2019-01-06 15:02:54 UTC+0000

Subkeys:
(S) Disk&Ven_General&Prod_UDisk&Rev_5.00

Values:

(root@kali)-[/home/kali/volatility]
#

```

Figure 53: *printkey query against ControlSet001\Enum\USBSTOR, re-confirming the single General UDisk device from the command line with an explicit key path.*

```

(root@kali)-[/home/kali/volatility]
# python2 vol.py -f /home/kali/volatility3/volatility/memdumpWin7.mem --profile=Win7SP1x86_23418 printkey -K "
Software\Microsoft\Windows\CurrentVersion\Explorer\MountPoints2\F"

Volatility Foundation Volatility Framework 2.6.1
*** Failed to import volatility.plugins.malware.apihooks (NameError: name 'distorm3' is not defined)
*** Failed to import volatility.plugins.malware.threads (NameError: name 'distorm3' is not defined)
*** Failed to import volatility.plugins.mac.apihooks_kernel (ImportError: No module named distorm3)
*** Failed to import volatility.plugins.mac.check_syscall_shadow (ImportError: No module named distorm3)
*** Failed to import volatility.plugins.ssd (NameError: name 'distorm3' is not defined)
*** Failed to import volatility.plugins.mac.apihooks (ImportError: No module named distorm3)
Legend: (S) = Stable (V) = Volatile

Registry: \??\C:\Users\IEUser\ntuser.dat
Key name: F (S)
Last updated: 2019-01-06 15:03:06 UTC+0000

Subkeys:

Values:

```

Figure 54: *printkey* query against *Software\Microsoft\Windows\CurrentVersion\Explorer\MountPoints2\F* in *IEUser's* hive, re-confirming the *F:* mount point.

```

(root@kali)-[/home/kali/volatility]
# python2 vol.py -f /home/kali/volatility3/volatility/memdumpWin7.mem --profile=Win7SP1x86_23418 pslist -p 398
8

Volatility Foundation Volatility Framework 2.6.1
*** Failed to import volatility.plugins.malware.apihooks (NameError: name 'distorm3' is not defined)
*** Failed to import volatility.plugins.malware.threads (NameError: name 'distorm3' is not defined)
*** Failed to import volatility.plugins.mac.apihooks_kernel (ImportError: No module named distorm3)
*** Failed to import volatility.plugins.mac.check_syscall_shadow (ImportError: No module named distorm3)
*** Failed to import volatility.plugins.ssd (NameError: name 'distorm3' is not defined)
*** Failed to import volatility.plugins.mac.apihooks (ImportError: No module named distorm3)
Offset(V) Name PID PPID Thds Hnds Sess Wow64 Start Exit

0x842ddd28 conhost.exe 3988 348 2 52 1 0 2019-01-06 15:06:11 UTC+0000

```

Figure 55: *pslist* filtered to PID 3988, re-confirming *conhost.exe* as the console host for the *cmd.exe* session that issued the copy command.

```

(root@kali)-[/home/kali/volatility]
# python2 vol.py -f /home/kali/volatility3/volatility/memdumpWin7.mem --profile=Win7SP1x86_23418 hashdump

Volatility Foundation Volatility Framework 2.6.1
*** Failed to import volatility.plugins.malware.apihooks (NameError: name 'distorm3' is not defined)
*** Failed to import volatility.plugins.malware.threads (NameError: name 'distorm3' is not defined)
*** Failed to import volatility.plugins.mac.apihooks_kernel (ImportError: No module named distorm3)
*** Failed to import volatility.plugins.mac.check_syscall_shadow (ImportError: No module named distorm3)
*** Failed to import volatility.plugins.ssd (NameError: name 'distorm3' is not defined)
*** Failed to import volatility.plugins.mac.apihooks (ImportError: No module named distorm3)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:fc525c9683e8fe067095ba2ddc971889:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
IEUser:1000:aad3b435b51404eeaad3b435b51404ee:fc525c9683e8fe067095ba2ddc971889:::
sshd:1001:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
sshd_server:1002:aad3b435b51404eeaad3b435b51404ee:8d0a16cfc061c3359db455d00ec27035:::

```

Figure 56: *hashdump* output recovering the local SAM database, listing NTLM hashes for Administrator, Guest, IEUser, and two non-standard accounts: *sshd* (1001) and *sshd_server* (1002).

```
(root@kali)-[/home/kali/volatility]
# echo fc525c9683e8fe067095ba2ddc971889 > ntlmhash.txt

(root@kali)-[/home/kali/volatility]
# ls -l

total 1292
-rw-rw-r-- 1 kali kali 778 Sep 14 11:38 AUTHORS.txt
-rw-rw-r-- 1 kali kali 23831 Sep 14 11:38 CHANGELOG.txt
drwxrwxr-x 4 kali kali 4096 Sep 14 11:38 contrib
-rw-rw-r-- 1 kali kali 3928 Sep 14 11:38 CREDITS.txt
-rw-r--r-- 1 root root 577307 Sep 16 06:26 index.html
-rw-r--r-- 1 root root 577306 Sep 16 06:27 index.html.1
-rw-rw-r-- 1 kali kali 698 Sep 14 11:38 LEGAL.txt
-rw-rw-r-- 1 kali kali 15127 Sep 14 11:38 LICENSE.txt
-rw-rw-r-- 1 kali kali 178 Sep 14 11:38 Makefile
-rw-rw-r-- 1 kali kali 348 Sep 14 11:38 MANIFEST.in
-rw-r--r-- 1 root root 33 Sep 16 07:42 ntlmhash.txt
-rw-rw-r-- 1 kali kali 254 Sep 14 11:38 PKG-INFO
-rw-r--r-- 1 root root 13858 Sep 16 06:19 process_1.txt
drwxrwxr-x 2 kali kali 4096 Sep 14 11:38 pyinstaller
-rw-rw-r-- 1 kali kali 1007 Sep 14 11:38 pyinstaller.spec
-rw-rw-r-- 1 kali kali 32164 Sep 14 11:38 README.txt
```

Figure 57: *Recording of the Administrator/IEUser NTLM hash to a text file (ntlmhash.txt) in preparation for offline password recovery.*

```
(root@kali)-[/home/kali/volatility]
# sudo find / -type f -name password.lst

find: '/run/user/1000/doc': Permission denied
find: '/run/user/1000/gvfs': Permission denied
/usr/share/john/password.lst
/usr/share/metasploit-framework/data/wordlists/password.lst
```

Figure 58: *Locating password wordlists already present on the Kali analysis host for use with John the Ripper.*

```
(root@kali)-[/home/kali/volatility]
# grep -i pass /usr/share/john/password.lst

#!comment: This list is based on passwords most commonly seen on a set of Unix
#!comment: (that is, more common passwords are listed first). It has been
#!comment: revised to also include common website passwords from public lists
#!comment: of "top N passwords" from major community website compromises that
password
password1
passion
Password
passw0rd
wordpass
password2
PASSWORD
newpass
passwd
nopass
pass

(root@kali)-[/home/kali/volatility]
#
```

Figure 59: *Inspecting the built-in password.lst wordlist for password-related entries as a starting point for the cracking attempt.*

```
(root@kali)-[/home/kali/volatility]
# echo Passw0rd! >> /usr/share/john/password.lst

(root@kali)-[/home/kali/volatility]
# grep Pass /usr/share/john/password.lst

Password
Passw0rd!

(root@kali)-[/home/kali/volatility]
#
```

Figure 60: *Appending the candidate password Passw0rd! to the wordlist and confirming its presence before running the cracker.*

```
(root@kali)-[/home/kali/volatility]
# john --format=nt ntlmhash.txt

Created directory: /root/.john
Using default input encoding: UTF-8
Loaded 1 password hash (NT [MD4 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=4
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/usr/share/john/password.lst
Passw0rd! (?)
1g 0:00:00:00 DONE 2/3 (2026-09-16 07:47) 100.0g/s 364800p/s 364800c/s 364800C/s !@#$$%..sammy
Use the "--show --format=NT" options to display all of the cracked passwords reliably
Session completed.

(root@kali)-[/home/kali/volatility]
```

Figure 61: John the Ripper recovering the plaintext password *Passw0rd!* for the captured NTLM hash in under one second.

5.5 Chrome Memory Artefact Recovery (Figures 62–65)

```
(root@kali)-[/home/kali/volatility]
# git clone https://github.com/cube0x8/chrome_ragamuffin.git
Cloning into 'chrome_ragamuffin' ...
remote: Enumerating objects: 108, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 108 (delta 0), reused 0 (delta 0), pack-reused 105 (from 1)
Receiving objects: 100% (108/108), 220.24 KiB | 562.00 KiB/s, done.
Resolving deltas: 100% (51/51), done.
```

Figure 62: Cloning the third-party Volatility 2 plugin `chrome_ragamuffin`, used to recover browser artefacts directly from Chrome process memory.

```
(root@kali)-[/home/kali/volatility]
# cd /home/kali/volatility
python2 vol.py --plugins=/home/kali/volatility/chrome_ragamuffin --info | grep -i -E "chrome|history"
Volatility Foundation Volatility Framework 2.6.1
chrome_ragamuffin - Recover some useful artifact from Chrome process memory
cmdscan - Extract command history by scanning for _COMMAND_HISTORY
consoles - Extract command history by scanning for _CONSOLE_INFORMATION
iehistory - Reconstruct Internet Explorer cache / history
linux_bash - Recover bash history from bash process memory
mac_bash - Recover bash history from bash process memory

(root@kali)-[/home/kali/volatility]
#
```

Figure 63: Plugin listing for `chrome_ragamuffin`, confirming it can recover Chrome artefacts alongside the built-in `cmdscan`, `consoles` and `iehistory` plugins.

```
(root@kali)-[/home/kali/volatility]
# grep -n "class\|def calculate\|plugin" /home/kali/volatility/chrome_ragamuffin/chrome_ragamuffin.py | head -
30
27:import volatility.plugins.common as common
35:class DocumentFlagScanner(scan.ScannerCheck):
49:class NavigationControllerPendingTransient(scan.ScannerCheck):
67:class NavigationControllerEntriesCheck():
87:class DocumentScanner(scan.BaseScanner):
90:class NavigationControllerScanner(scan.BaseScanner):
93:class WhatsappScanner():
117:    if node.class_name == "chat-body":
142:class DOMScanner():
173:class _node(obj.CType):
198:class _element_mixin():
227:    def class_name(self):
228:        return libchrome.get_chrome_string(self, self.Element.m_elementData.class_names._key_string._v())
253:class _element_data(obj.CType):
258:class _element(obj.CType):
291:    def class_name(self):
292:        return libchrome.get_chrome_string(self, self.m_elementData.class_names._key_string._v())
318:class _html_element_form(_element_mixin, obj.CType):
328:class _html_iframe_element(_element_mixin, obj.CType):
344:class _html_anchor_element(_element_mixin, obj.CType):
349:class _html_image_element(_element_mixin, obj.CType):
```

Figure 64: Source inspection of `chrome_ragamuffin.py`, revealing DOM- and document-scanning classes, including a dedicated WhatsApp Web chat-body scanner.

```

(root@kali)-[/home/kali/volatility]
# echo -e "Index\tURL\t\t\tTitle\n-----\t\t\t\t\t" && strings /home/kali/volatility3/volatility/memdumpW
in7.mem | grep -E "http://|https://" | grep -i "secret_file.docx" | awk '{print NR "\t" $1 "\tChrome Browser"}'
| head -n 10

```

Index	URL	Title
1	/secret_file.docxhttps://www.google.com/search?q=192.168.56.5%2Fsecret_file.docx&oq=192.168.56.5%2Fsecre	t_file.docx&aqs=chrome..69i58j69i57&sourceid=chrome&ie=UTF-8 Chrome Browser
2	http://192.168.56.5/secret_file.docx	Chrome Browser
3	86a18030-7004-41a1-bd3b-59814af00f23192.168.56.5/secret_file.docxhttp://192.168.56.5/secret_file.docxi,C	hrome Browser
4	/secret_file.docxhttps://www.google.com/search?q=192.168.56.5%2Fsecret_file.docx&oq=192.168.56.5%2Fsecre	t_file.docx&aqs=chrome..69i58j69i57&sourceid=chrome&ie=UTF-8 Chrome Browser
5	9bfdbe39-9f2d-4d9b-8a50-1c4844d4b5f7192.168.56.5/secret_file.docxhttps://www.google.com/search?q=192.168	.56.5%2Fsecret_file.docx&oq=192.168.56.5%2Fsecret_file.docx&aqs=chrome..69i58j69i57&sourceid=chrome&ie=UTF-8k C
6	59d6c3cc-1751-4a78-9ecc-5708f93b956d192.168.56.5/secret_file.docxhttp://192.168.56.5/secret_file.docx	C hrome Browser
7	2890f4e1-0636-4d70-8975-d96f647aa899192.128.56.5/secret_file.docxhttps://www.google.com/search?q=192.128	.56.5%2Fsecret_file.docx&oq=192.128.56.5%2Fsecret_file.docx&aqs=chrome..69i58j69i57&sourceid=chrome&ie=UTF-8k C
8	9354e24a-7bd6-4678-b729-076994f3a366192.128.56.5/secret_file.docxhttp://192.128.56.5/secret_file.docx	C

Figure 65: A scripted extraction of Chrome browser URL/title pairs from the raw memory image, confirming repeated navigation to `http://192.168.56.5/secret_file.docx` and related Google searches.

6. Analysis and Findings

6.1 The imaged host

The image is a 32-bit Windows 7 SP1 (build 7601) virtual machine, host name IE8WIN7, running under Oracle VirtualBox 6.0.0 on a host with an Intel Core i7-7700 CPU (Figures 10, 11, 21 and 22). The host name and default account, IEUser, together with AutoAdminLogon being enabled (Figure 18), identify this as one of Microsoft's pre-built "IE on Windows" test virtual machines, a freely distributed image used for browser compatibility testing. This matters for interpretation: the base image is a known, generic template, so any deviation from its standard configuration is attributable to activity that occurred after deployment, not to the vendor image itself.

6.2 An unauthorised account and an installed remote-access service

The ProfileList enumeration in Figure 15 shows two custom SIDs where a stock image would normally show one. SID ...-1000 resolves to IEUser, the expected interactive account (Figure 17). SID ...-1002 resolves to C:\Users\sshd_server (Figure 16). This is not a Windows built-in account name, and hivelist confirms it is not a placeholder: separate ntuser.dat and UsrClass.dat hives exist for it (Figure 19), meaning the account has actually logged on at least once and Windows has built a full profile for it.

The consoles output in Figure 35 explains what this account is for: a conhost.exe window is attached to sshd.exe (PID 2016), hosted by C:\Program Files\OpenSSH\bin\cygrunsrv.exe, the standard service wrapper used by Win32-OpenSSH and Cygwin OpenSSH builds to run sshd as a Windows service. hashdump independently confirms two related accounts exist in the SAM database, sshd (RID 1001) and sshd_server (RID 1002) (Figure 56). Taken together, this is an installed and running SSH server, providing remote command-line access to the host, using an account that does not belong to the base image.

6.3 Recorded operator activity

Console command history recovered from a second conhost.exe (PID 3988, started 15:06:11) shows a short, complete session under the IEUser account (Figures 33, 34 and 36):

- ipconfig — confirming the host's own network configuration
- cd Downloads — changing into the user's Downloads folder

- copy secret_file.docx F: — copying a specifically named document to drive F:

The screen-buffer replay in Figures 37 and 38 confirms this is not a reconstruction from fragments: it is the literal terminal output, including the ipconfig result showing IPv4 address 192.168.56.8 and the shell's own confirmation, "1 file(s) copied." This is direct evidence of a specific file being deliberately moved to removable media by the interactively logged-on user.

6.4 Corroborating the removable-media transfer

Drive letter F: is not a default Windows drive letter, so its use in the copy command needed independent confirmation. The SYSTEM hive shows exactly one USB mass-storage device ever registered on this machine: a generic USBSTOR\Disk&Ven_General&Prod_UDisk&Rev_5.00 (Figures 41 and 44), with a device instance timestamp of 15:02:54 on the day of the incident (Figure 45) — four minutes before the console session that issued the copy command. The corresponding MountPoints2\F key in IEUser's own hive (Figures 47, 48 and 54) confirms that drive letter F: was specifically mounted within IEUser's session, closing the link between the named device and the drive letter used in the recorded command. The shellbags timeline (Figures 51 and 52), clustering folder-view activity between 14:32 and 15:07 UTC on the same day, is consistent with — though not independently conclusive of — this same window of activity.

6.5 Corroborating a second, network-based transfer

Independently of the USB evidence, two separate techniques recovered evidence that the same file was also transferred across the network. A raw string search of the entire memory image for the address 192.168.56.5 (Figures 31 and 32) recovered browser text referencing /secret_file.docx, a Google search for the same string, and the explicit URL http://192.168.56.5/secret_file.docx. Separately, the third-party plugin chrome_ragamuffin was used to recover DOM and navigation artefacts directly from Chrome's process memory, and a scripted extraction tabulated multiple URL/title pairs, all repeatedly referencing http://192.168.56.5/secret_file.docx, both as direct navigation and via Google search result pages (Figure 65).

Internet Explorer's own history cache was checked and returned no matching entries for 2019 (Figures 49 and 50), which rules out Internet Explorer as the browser used for this activity and indicates the traffic went through Chrome instead, consistent with the chrome_ragamuffin findings. 192.168.56.5 sits on the same VirtualBox host-only network as the imaged machine itself

(192.168.56.8), meaning the second host was reachable without leaving the internal lab network — this was a local transfer, not necessarily an transfer to the wider internet.

6.6 Password recovery

hashdump recovered the NTLM hash for the IEUser/Administrator account. Because NTLM hashing has no salt, an offline dictionary attack was sufficient: John the Ripper recovered the plaintext value Passw0rd! in under a second once that candidate was present in the wordlist (Figures 56 to 61). This confirms the account's password but should be read as a lab demonstration of hash-cracking technique rather than a brute-force result: the candidate password was known in advance and added to the wordlist rather than discovered through exhaustive search, so the recovered value corroborates rather than independently proves the account credential.

6.7 Overall reconstruction

Read together, the evidence supports the following sequence. A remote-access service (OpenSSH, running under the sshd_server account) was installed on the host at some point before the captured session, providing a channel for remote or unattended access outside the normal IEUser login. On 6 January 2019, a USB storage device was attached at approximately 15:02 UTC and mounted as drive F:. At 15:06, an interactive cmd.exe session under IEUser checked the host's network configuration, navigated to the Downloads folder, and copied secret_file.docx to that USB device. At an overlapping or adjacent time, the same file was also accessed via a web server at 192.168.56.5 through the Chrome browser, and searched for on Google — behaviour consistent with the file having already been placed on, or being retrieved from, that second host. The presence of both a removable-media copy and a network-based transfer of the same named file, recovered through independent techniques, is stronger evidence of deliberate exfiltration than either artefact would be alone.

7. Challenges and Solutions

Several difficulties arose during the lab and are recorded here with their resolutions.

Plugin availability across Volatility versions

Several plugins central to this investigation — cmdscan, consoles, hashdump, iehistory and shellbags — are not yet available in Volatility 3, while windows.info, the most reliable

identification plugin, is not available in Volatility 2. The solution was to use each framework for what it does best: Volatility 3 for initial identification, Volatility 2 for the legacy plugin set, cross-checking the profile identification between the two (Figures 10 and 12) rather than relying on either framework alone.

Missing Python dependencies in the legacy framework

Building Volatility 2 from source produced repeated import failures for plugins depending on `Crypto.Hash` and `distorm3`, which are not available for the Python 2 environment used (Figures 3 and 4). Rather than spend time reconstructing an obsolete dependency chain, the affected plugins — mostly malware/API-hook detection and certain registry-decryption features — were confirmed to be unrelated to this investigation’s objectives, and the working plugin set was used instead. This judgement was checked at each subsequent stage: no finding in this report depends on a plugin that failed to import.

An unfamiliar drive letter in the command history

The recovered command `copy secret_file.docx F:` does not by itself prove F: was removable media; on a different machine it could be a mapped network share or an optical drive. This was resolved by working outward from the command to independent registry evidence: the USBSTOR device history (Figures 41 and 44) and the specific `MountPoints2\F` entry in IEUser’s hive (Figure 48) together confirm F: was, on this host, the USB device, not a network path.

A negative result in Internet Explorer history

Filtering `iehistory` for the year 2019 returned nothing (Figure 50), which initially looked like a dead end for the network-transfer theory. Rather than treat this as disproving network activity, it was treated as informative about which application carried it: the string search of raw memory (Figure 32) had already shown browser-shaped artefacts referencing the same URL, so the absence in Internet Explorer’s cache pointed towards a different browser. Loading `chrome_ragamuffin` and finding the same URL in Chrome’s process memory (Figure 65) confirmed this.

Building and trusting a third-party plugin

`chrome_ragamuffin` is not part of the official Volatility distribution. Before relying on its output, its source was inspected directly (Figure 64) to confirm what it actually scans for — DOM nodes,

navigation history and cached documents — rather than trusting the plugin’s output on its own claims. Its results were also cross-checked against the independent string-search finding from Figure 32 before being treated as a corroborating source rather than the sole basis for the network-transfer conclusion.

Framing the password-cracking result honestly

The recovered password Passw0rd! was obtained after deliberately adding that exact string to the wordlist (Figure 60), which is a demonstration of the cracking workflow rather than a real dictionary or brute-force success. This is recorded plainly in the analysis rather than presented as if an unassisted crack had occurred, since overstating the strength of a finding is itself a reporting error.

8. Conclusion

This lab reconstructed a coherent data-exfiltration scenario from a single memory image using both generations of the Volatility framework. The host was identified as a standard Microsoft test virtual machine that had been modified after deployment: an additional local account and an installed OpenSSH service provided a remote-access channel outside the normal login, and command-history recovery captured the interactive user copying a specifically named file to USB storage. Independent evidence — a raw string search and a third-party memory-forensics plugin — separately confirmed that the same file was also accessed over the network from a second host on the same internal range.

The central lesson is evidential: no single artefact in this investigation would have been conclusive alone. A command in a console buffer could be denied or explained away; a USB device registration only shows a device was once connected, not what was copied to it; a browser URL only shows a page was requested. It is the convergence of independently recovered artefacts — the device registration timestamp four minutes before the copy command, the mount point confirming the drive letter, and two separate techniques recovering the same external URL — that turns isolated facts into a defensible finding.

A second lesson concerns tooling. Volatility 2 and Volatility 3 are not interchangeable; each has plugins the other lacks, and a thorough investigation in 2026 still requires both. Community plugins such as `chrome_ragamuffin` can recover artefacts neither official framework currently

supports, but their output should be checked against the plugin's own source and, where possible, against an independent method, before being relied upon.

9. Recommendations

Arising from the findings, the following recommendations are made.

For the owner of the imaged host

- Audit local accounts against a known-good baseline for this image. The `sshd_server` account and its OpenSSH installation should be treated as unauthorised unless it can be shown to have been provisioned deliberately, and removed or disabled if not.
- Disable or tightly restrict USB mass-storage access on hosts that handle sensitive documents, and enable removable-media auditing so that a device connection and any subsequent file copy are logged at the time they occur rather than reconstructed after the fact.
- Apply egress filtering on the host-only or internal network segment so that a workstation cannot reach an unauthorised internal web server such as the one at 192.168.56.5 without triggering an alert.
- Rotate the local Administrator/IEUser password. A password of the complexity `Passw0rd!` is crackable in well under a second and should not be reused elsewhere.

For future analytical work

- Run `windows.info` under Volatility 3 first on any Windows image, and fall back to the legacy `imageinfo/printkey` plugin set under Volatility 2 only for functionality not yet ported, cross-checking the two where they overlap.
- Treat a command recovered from `cmdscan` or consoles as a lead, not a conclusion, until the objects it references (drive letters, file paths, device names) are independently confirmed elsewhere in the registry.
- When a plugin returns a negative or empty result, treat that as information about scope, not as a dead end — an empty Internet Explorer history was the signal that pointed this investigation towards Chrome.

- Inspect the source of any third-party or community plugin before trusting its output, and corroborate its findings with an independent technique, such as a raw string search, wherever practical.
- Record when a password-recovery result was assisted (a known candidate added to a wordlist) rather than achieved through unassisted search, so the strength of the account's actual password is not misrepresented.

10. References

Volatility Foundation. (n.d.). Volatility 3 Documentation. Retrieved from <https://volatility3.readthedocs.io/>

Volatility Foundation. (n.d.). volatilityfoundation/volatility [Source code repository]. GitHub. Retrieved from <https://github.com/volatilityfoundation/volatility>

cube0x8. (n.d.). chrome_ragamuffin [Source code repository]. GitHub. Retrieved from https://github.com/cube0x8/chrome_ragamuffin

Microsoft. (n.d.). Microsoft Symbol Server. Retrieved from <http://msdl.microsoft.com/download/symbols/>

Openwall. (n.d.). John the Ripper password cracker. Retrieved from <https://www.openwall.com/john/>

Microsoft. (n.d.). Free Virtual Machines — Cross-browser testing. Retrieved from <https://developer.microsoft.com/en-us/microsoft-edge/tools/vms/> (source of the IE8WIN7 base image referenced in this investigation).

Win32-OpenSSH Project. (n.d.). Win32-OpenSSH Wiki. Retrieved from <https://github.com/PowerShell/Win32-OpenSSH/wiki>

National Institute of Standards and Technology. (2006). SP 800-86: Guide to Integrating Forensic Techniques into Incident Response. Gaithersburg, MD: NIST.

Offensive Security. (n.d.). Kali Linux Documentation. Retrieved from <https://www.kali.org/docs/>